

Order Demo Program

Java Lab Record

1. Aim

To develop a Java program to demonstrate classes, objects, constructors, inheritance, and method calls using Customer, Order, SpecialOrder, and NormalOrder classes.

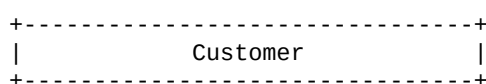
2. Step-by-Step Procedure

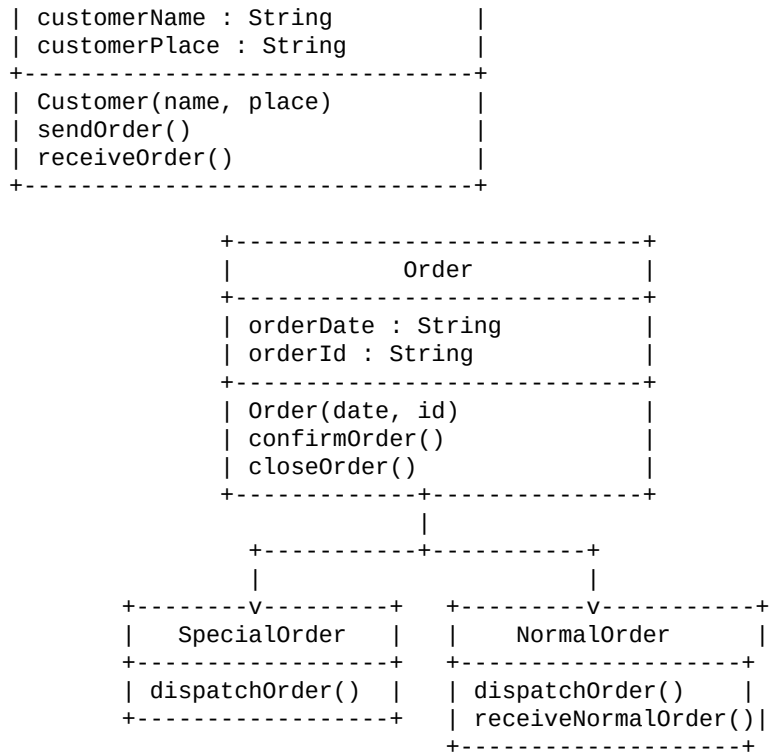
1. Create a Customer class with customerName and customerPlace as data members.
2. Create a constructor to initialize the customer details.
3. Create sendOrder() and receiveOrder() methods to display customer activities.
4. Create an Order class with orderDate and orderId as data members.
5. Create a constructor to initialize the order details.
6. Create confirmOrder() and closeOrder() methods.
7. Create SpecialOrder as a subclass of Order and use super() to call the parent constructor.
8. Create dispatchOrder() in the SpecialOrder class.
9. Create NormalOrder as a subclass of Order.
10. Create dispatchOrder() and receiveNormalOrder() methods in NormalOrder.
11. Create objects of Customer, SpecialOrder, and NormalOrder in the main() method.
12. Display customer information, order details, and customer activities.
13. Stop the program.

3. Algorithm

1. Start.
2. Create a Customer object with name and place.
3. Create a SpecialOrder object with date and order ID.
4. Create a NormalOrder object with date and order ID.
5. Display the customer information.
6. Call confirmOrder(), dispatchOrder(), and closeOrder() for the SpecialOrder.
7. Call confirmOrder(), dispatchOrder(), receiveNormalOrder(), and closeOrder() for the NormalOrder.
8. Call sendOrder() and receiveOrder() for the Customer.
9. Stop.

4. UML Class Diagram





5. Code

```

class Customer {
    String customerName;
    String customerPlace;

    Customer(String customerName,String customerPlace) {
        this.customerName=customerName;
        this.customerPlace=customerPlace;
    }

    void sendOrder() {
        System.out.println(customerName+" has sent the order.");
    }

    void receiveOrder() {
        System.out.println(customerName+" has received the order.");
    }
}

class Order {
    String orderDate;
    String orderId;

    Order(String orderDate,String orderId) {
        this.orderDate=orderDate;
        this.orderId=orderId;
    }

    void confirmOrder() {
        System.out.println("Order "+orderId+" confirmed.");
    }

    void closeOrder() {
        System.out.println("Order "+orderId+" closed.");
    }
}

class SpecialOrder extends Order {
    SpecialOrder(String date,String id) {
        super(date,id);
    }
    void dispatchOrder() {

```

```

        System.out.println("Special Order "+orderId+" dispatched.");
    }
}

class NormalOrder extends Order {
    NormalOrder(String date,String id) {
        super(date,id);
    }

    void dispatchOrder() {
        System.out.println("Normal Order "+orderId+" dispatched.");
    }

    void receiveNormalOrder() {
        System.out.println("Normal Order "+orderId+" received.");
    }
}

public class OrderDemo {
    public static void main(String[] args) {
        Customer c1=new Customer("Ravi","Tagarapuvalasa");

        SpecialOrder s1=new SpecialOrder("11-08-2026","S101");

        NormalOrder n1=new NormalOrder("11-08-2026","N101");

        System.out.println("Customer Information");
        System.out.println("Customer Name: "+c1.customerName);
        System.out.println("Place: "+c1.customerPlace);

        System.out.println("\nSpecial Order Details");
        s1.confirmOrder();
        s1.dispatchOrder();
        s1.closeOrder();

        System.out.println("\nNormal Order Details");
        n1.confirmOrder();
        n1.dispatchOrder();
        n1.receiveNormalOrder();
        n1.closeOrder();

        System.out.println("\nCustomer Activities");
        c1.sendOrder();
        c1.receiveOrder();
    }
}

```

6. Sample Output

The following output is obtained by executing the updated OrderDemo program.

Customer Information

Customer Name: Ravi

Place: Tagarapuvalasa

Special Order Details

Order S101 confirmed.

Special Order S101 dispatched.

Order S101 closed.

Normal Order Details

Order N101 confirmed.

Normal Order N101 dispatched.

Normal Order N101 received.

Order N101 closed.

Customer Activities

Ravi has sent the order.

Ravi has received the order.